CampusConnect

System Architecture, Database Design & Workflow Diagrams

Arya College of Engineering and IT — Initial Campus

Product direction	A real, college-specific campus platform starting with Arya College and designed for future multi-college expansion.
Frontend	React + Vite
Backend	Spring Boot + Java REST APIs
Authentication	JWT-based authentication with role-based authorization
Database	MySQL with college-scoped records

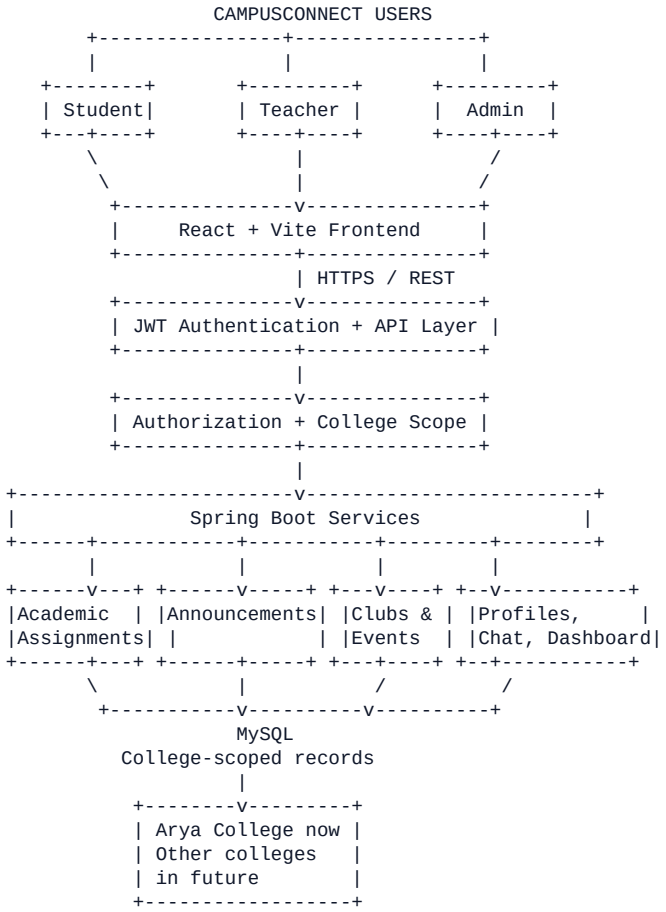
Purpose

This document consolidates the proposed CampusConnect architecture and core workflows into one reference PDF. It is intended for planning, implementation, team discussion and project documentation.

Important: These diagrams describe the proposed target design. They are not proof that every module or table has already been implemented.

1. System Architecture

The frontend presents role-specific experiences. Requests pass through authentication and authorization, then into application services. Services persist college-scoped records in MySQL. Arya College is the initial deployment scope; additional colleges may be onboarded later without mixing their data.

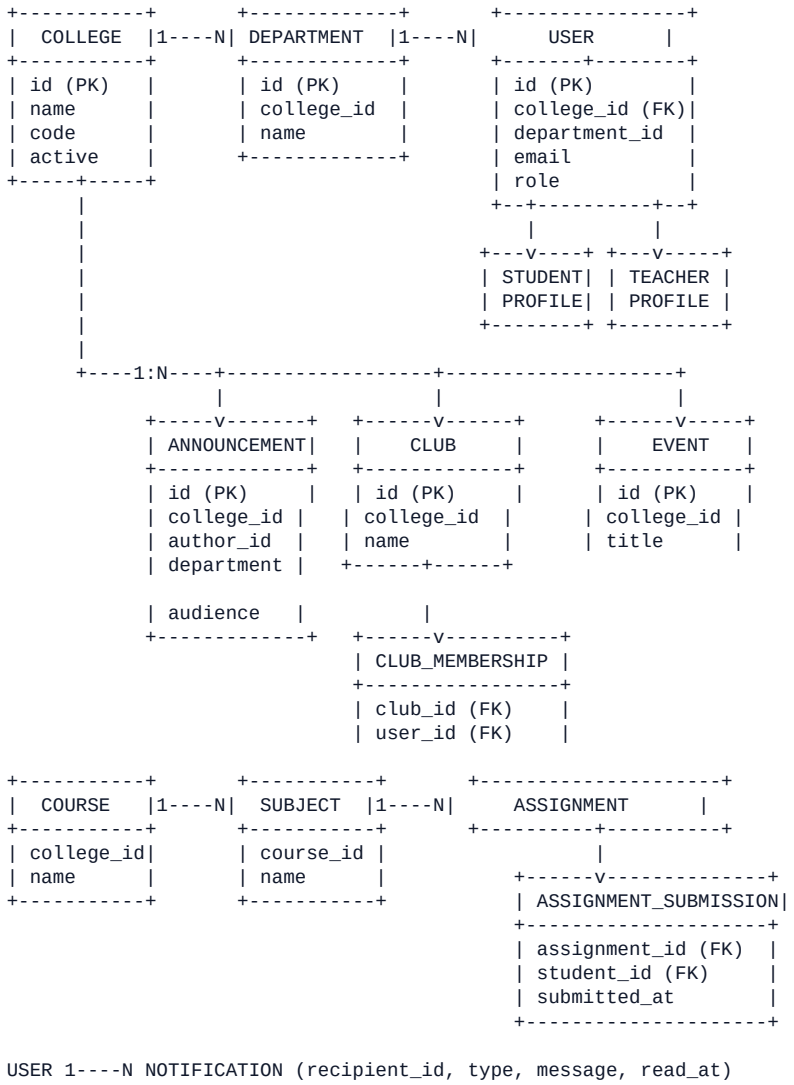


Key architecture principles

- Frontend route visibility is not security; the backend must enforce user roles and college scope.
- Store college association on relevant records, directly or through a well-defined parent entity.
- Keep secrets such as database passwords and JWT signing keys in environment variables, never in Git.
- Use migrations and tests before changing production database schemas.

2. Proposed Database ER Diagram

The ER diagram describes a future-ready relational model. The final schema must be reconciled with the existing entities and tables before any migration is applied.



Suggested entities and relationships

- College: institution identity and active status.
- Department: belongs to one college; user department associations should be validated against the same college.
- User and profiles: authentication identity and role-specific student/teacher details.
- Announcements, clubs and events: scoped to a college, with author/owner references and optional department or audience targeting.
- Courses, subjects, assignments and submissions: connect academic work to the relevant college, course and student.
- Notifications: have an explicit recipient and reference the triggering event where practical.

Design note: Relationship cardinalities and optionality should be finalized from real requirements. For example, a teacher may teach multiple subjects, so a teaching-assignment relationship may be needed.

3. Login and Role-Based Dashboard Workflow

After credentials are verified, the application resolves the user's role and loads only authorized records for the user's college and department. Invalid credentials should not reveal sensitive account information.

Dashboard responsibilities

- Student: relevant notices, upcoming assignments, class resources, club/event information and personal notifications.
- Teacher: classes/subjects, assignment management, relevant student submissions and authorized notice publishing.
- College admin: account and content management, institution-wide notices, moderation and administrative oversight.
- All roles: API responses must be checked server-side; a user must not gain access by editing browser storage or request parameters.

4. Announcement Publishing and Viewing Workflow

An announcement is created by an authorized teacher or administrator, validated, persisted with its college scope, and returned only to users who match its audience and permissions.

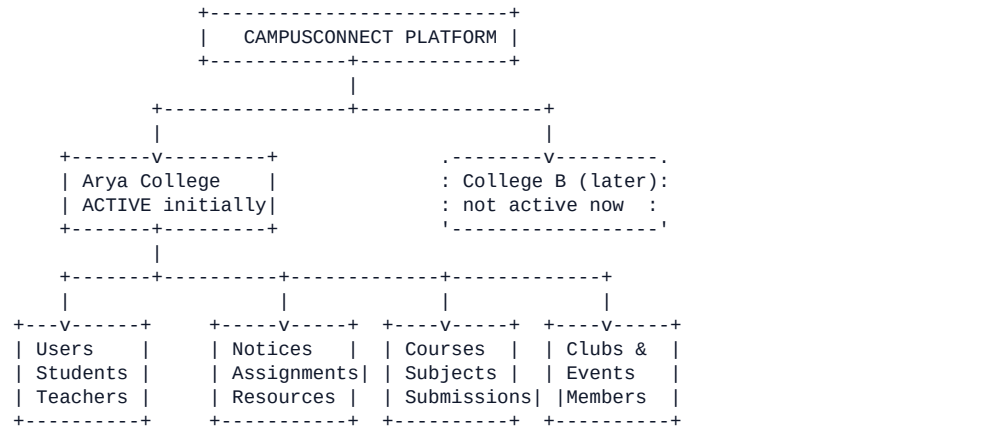
```
[Teacher/Admin opens Announcements]
                |
                v
     [Enter title, content, category, audience]
                |
                v
     {Authenticated and authorized?} -- No --> [401/403 response]
                | Yes
                v
           {Input valid?} -- No --> [400 validation response]
                | Yes
                v
     [Determine college scope on server]
                |
                v
     [Save announcement in MySQL]
                |
                v
     [Apply audience/department visibility rules]
                |
                v
     [Return relevant notices through API]
                |
                v
     [Student views notice in dashboard/feed]

Editing/deleting:
- Author may edit/delete own notice if permitted by policy.
- College admin may manage notices within their authorized college.
- Every read/write query must enforce the appropriate college boundary.
```

Important error handling

- Return clear client-safe errors for invalid input, missing records and insufficient permissions.
- Log diagnostic details on the server, but do not expose stack traces or credentials to the browser.
- Test duplicate, empty, too-long and unauthorized requests, as well as audience filtering.
- A successful UI click is not proof of a successful save; verify the API response and persisted record.

5. Multi-College Expansion and Data Isolation

Arya College is the only initial target. The architecture can be multi-college-ready without onboarding or displaying fabricated data for other institutions.




```
Mandatory isolation rule:
A request must be authorized for the current user AND the record's college.
Never trust a client-supplied college_id by itself.
Use scoped queries, service-level authorization and appropriate database constraints.
```

Data isolation checklist

- Derive the user's college from the authenticated identity, not an untrusted request body.
- Ensure create, read, update, delete, search, counts and dashboard aggregates apply college scope.
- Validate that department, course, subject, author and audience references belong to the same college where required.
- Add automated cross-college access tests before onboarding a second institution.
- Decide whether administrators are college-scoped or platform-wide; do not silently grant global access.

6. CampusConnect Module Map

Module	Student experience	Teacher/Admin capability
Dashboard	Relevant notices, deadlines, events, quick links	Role-aware summary and authorized actions
Academics	Subjects, class resources, assignments	Manage subjects/resources where authorized
Assignments	View due dates and submit work	Create, edit, review submissions
Announcements	College/department notices	Create, pin, target and manage notices
Clubs & Events	Discover clubs, register for events	Manage membership and event details
Communities & Chat	Authorized class/club discussions	Moderation and group management
Profiles	Manage personal and academic details	Maintain verified institutional records
Administration	Not generally exposed as admin tools	Manage users, departments, content, access

7. Recommended Implementation Roadmap

Phase 1 — Audit

Inventory real APIs, database entities, mock data, placeholders, role rules and existing tests. Do not change code during the audit.

Phase 2 — Data foundation

Agree on College and Department entities, define migration/backfill rules, and associate existing users/data with Arya College safely.

Phase 3 — Core academic workflow

Stabilize login/profile, announcements, subjects, assignments and submissions; replace mock cards with verified API data.

Phase 4 — Campus life

Implement real clubs, events, memberships, communities and notifications with role/college scoping.

Phase 5 — Admin and content

Add authorized management workflows and populate only verified Arya College information.

Phase 6 — Quality and deployment

Run unit/integration tests, cross-role and cross-college authorization tests, then deploy frontend/backend and verify production APIs.

Final principle

Build for Arya College now, design for multiple colleges later, and never trade data correctness or security for demo content.